

Audio vs Transcript — Quality Assessment

Two independent approaches to assessing speech-to-text quality, sharing a common repo. `approaches.md` explains why both exist.

	Approach 1	Approach 2
Needs a gold transcript?	Yes (audio + human transcript)	No (audio only)
How accuracy is judged	Whisper vs gold, LLM-judged	Whisper vs Deepgram consensus
Deterministic core	Signals + scoring	Alignment + agreement + tiers
Human review	Error report	Interactive spot-check UI

Which one should I use?

- **Approach 2** (two-engine consensus) when you *only* have audio and want to know, per segment, how confident both engines are — no gold transcript needed.
- **Approach 1** when you already have a trusted reference transcript and need to audit the STT output against it (e.g. verifying a dictation against source).

If you can get a gold transcript, Approach 1 gives a stronger verdict; Approach 2 is the choice for fully automated, reference-free QA. The two can be combined: run Approach 2 to flag risky spans, then verify them with Approach 1 if you also have the transcript.

Approach 1 — gold-reference comparison

Whisper transcribes the audio; the transcript is compared against a gold reference (`.pdf/.txt`) and graded for missing / incorrect / conflicting / hallucinated information.

```
cd approach_1
python -m venv .venv && source .venv/bin/activate # or reuse the repo venv
pip install -r requirements.txt
cp ../.env .env # provides DEEPSEEK_API_KEY
```

Config (via `.env`)

Variable	Default	Meaning
STT_MODEL_NAME	base	faster-whisper size
EVAL_MODEL_NAME	deepseek-v4-flash	LLM judge model
DEEPSEEK_API_KEY	—	required for the judge
EMBEDDINGS_ENABLED	true	use sentence-transformers signal

Variable	Default	Meaning
<code>SCORE_THRESHOLD</code>	<code>90</code>	below this -> status <code>Mismatch</code>

Run

Step 1 — generate the STT transcript (the model converts audio → text):

```
python -m approach_1.main transcribe path/to/audio.wav
```

- Runs faster-whisper (`STT_MODEL_NAME`, default `base`) on the audio.
- Saves the text to `approach_1/datasets/stt_generated_transcripts/<audio_stem>_stt.txt` and prints it to stdout.
- Reuses the cached transcript on repeat runs; add `--force-stt` to re-transcribe.

Step 2 — compare STT against the gold reference and get the report:

```
# (a) audio + gold transcript, one command (transcribes if not cached, then compares)
python -m approach_1.main evaluate path/to/audio.wav --gold path/to/gold.txt

# (b) compare two text files directly (no audio/STT needed)
python -m approach_1.main evaluate-text --gold path/to/gold.txt --candidate path/to/candidate.txt
```

Both print the full JSON report: overall score (0–100), status (`Match` / `Mismatch`), score breakdown, signals, and the categorized findings (missing / incorrect / conflicting / hallucinated).

Optionally set the STT model size before running:

```
STT_MODEL_NAME=small python -m approach_1.main transcribe path/to/audio.wav
```

Web UI (interactive):

```
python -m approach_1.main transcribe path/to/audio.wav # optional: pre-generate STT
uvicorn approach_1.api:app --reload --port 8000
```

Open <http://localhost:8000/>. Pages:

Path	Purpose
/	review stored evaluation runs (score, findings, transcripts) — the main page
/new	run a new evaluation (upload audio + gold transcript, click Evaluate)
/framework	synthetic test-cases used to verify the pipeline

Approach 2 — two-engine consensus

Two independent STT engines transcribe the same audio; their segments are aligned (time + text), word-diffed, and scored. Tiers decide what a human must review, and an interactive UI plays the exact span for each flagged segment.

```
source .venv/bin/activate
cd approach_2
pip install -r requirements.txt
```

Config (via **.env**)

Variable	Default	Meaning
WHISPER_MODEL	small	faster-whisper size (engine A)
DEEPGRAM_API_KEY	—	required for Deepgram Nova (engine B)
DEEPGRAM_MODEL	nova-3	Deepgram model
GLOSSARY_PATH	(empty)	optional domain-term wordlist (one per line)

Run

```
python -m approach_2.main transcribe          # transcribe all files in
dataset/audio
python -m approach_2.main transcribe audio-3.ogg # one file
python -m approach_2.main review              # evaluate stored
transcripts -> reports
python -m approach_2.main review audio-1      # one file
python -m approach_2.main evaluate audio-1    # transcribe (if missing) +
review in one step
```

Interactive review (play segments, mark correct/incorrect, correct text):

```
uvicorn approach_2.api:app --port 8000
```

Web pages (main page is **/**):

- `/` — audit stored reports (segments, agreement, verdicts)
- `/new` — run a new evaluation in the browser: upload audio, both engines transcribe it, the report is generated and appears on `/`
- `/framework` — synthetic test-cases used to verify the pipeline

Verdicts persist to `dataset/review/<name>/verdicts.json`; acceptance ($\geq 99\%$ on the reviewed sample) is recomputed live. CLI equivalents print the same status. See `approach_2/README.md` for the data layout, alignment notes, and tests.

Tiers

Confidence	Tier	Action
≥ 98	auto_accept	accept automatically
90–97	review_technical	review only if a glossary term is present
< 90	mandatory	must be human-verified

Tests

```
make tests
```

(`make` is available from the repo root; `make` alone prints a menu of every target, `make setup` does the one-time install.)

Data: real vs. synthetic test cases

- **Real evaluation data** lives per-approach under `approach_1/datasets/` (audio + manual `.pdf` golds + cached STT `.txt` + stored review runs in `datasets/review/`) and `approach_2/dataset/` (audio + per-engine transcripts + review reports). These are the actual files the applications run on.
- **Synthetic test-case dataset** is `dataset/test_cases.json`, a single JSON export of the shared error-injection catalog `testing/scenarios.py`. That catalog is the one source of truth consumed by **both** approaches' `pytest` suites (`approach_1/tests/test_scenarios.py`, `approach_2/tests/test_scenarios.py`) — so the submitted JSON never diverges from the tests. It contains the controlled cases (perfect match, number change, negation, missing word, garbled term, semantic-equivalent, spelling noise) with their expected categories.

The two are intentionally separate: real files demonstrate the system on actual recordings, while `test_cases.json` verifies that the evaluator detects and classifies known failure modes. The JSON is **not** read by any approach's runtime; regenerate it with `python -m testing.export_test_cases` if the catalog changes.